X728 UPS Auto Shutdown Installation Guide

What This Guide Will Help You Do

This guide will help you set up your Raspberry Pi so it automatically shuts down **both the Pi AND the UPS** safely when the power goes out. Think of it like a smart power protector that completely powers off your system to save battery!

What you'll need:

- Raspberry Pi 4B with X728 UPS board attached
- Computer keyboard and mouse
- Monitor connected to your Pi
- Internet connection
- About 30 minutes

What this does differently:

- ✅ **Complete Power Down:** Unlike basic scripts, this properly shuts down both Pi AND UPS
- ✅ **Battery Conservation:** UPS turns off completely to preserve battery life
- ✅ **Auto Power-On:** When power returns, the Pi automatically starts up
- ✅ **Based on Official Geekworm Methods:** Uses the same technique as their `x728off` command

---

Step 1: Check Your Hardware Setup

Before we start with software, let's make sure everything is connected properly:

1.1 Physical Connections

1. **Turn off your Pi** and unplug the power
2. **Check the X728 board** is properly connected to your Pi's GPIO pins
3. **Verify the jumpers** on the X728:
   - Make sure the "AUTO ON" jumper is connected (this allows auto power-on)
   - Make sure the "CHG EN" jumper is connected (this enables charging)
4. **Connect your power supply** to the X728 board (NOT directly to the Pi)
5. **Insert 18650 batteries** into the X728 if you haven't already

1.2 Power On Test

1. **Plug in the power** to the X728 board

2. **Press the power button** on the X728 to turn on your Pi
3. **Wait for the Pi to boot up** completely

✅ **Success Check:** Your Pi should boot normally and you should see the desktop

---

Step 2: Enable I2C Interface

The X728 board talks to your Pi using something called "I2C". We need to turn this on:

2.1 Open Configuration Tool

1. **Click on the terminal icon** on your desktop (it looks like a black square)
2. **Type this command** and press Enter:

```bash
sudo raspi-config
```

3. **Enter your password** if asked

2.2 Enable I2C

1. **Use arrow keys** to navigate to "Interface Options"
2. **Press Enter** to select it
3. **Navigate to "I2C"** and press Enter
4. **Select "Yes"** when asked "Would you like the ARM I2C interface to be enabled?"
5. **Select "OK"** to confirm
6. **Select "Finish"** to exit
7. **Select "Yes"** when asked to reboot

2.3 Verify I2C Works

After your Pi reboots:

1. **Open terminal again**
2. **Type this command** to check if I2C is working:

```bash
sudo i2cdetect -y 1
```

3. **Look for "36"** in the output - this is your X728 board!

✅ **Success Check:** You should see "36" somewhere in the grid that appears

---

Step 3: Install Required Software

Now let's install the tools our script needs:

3.1 Update Your System

1. **In terminal, type these commands** one by one:

```bash
sudo apt update
sudo apt upgrade -y
```

2. **Wait for updates to finish** (this might take a few minutes)

3.2 Install Required Libraries

```bash
sudo apt install -y python3-smbus python3-rpi.gpio gpiod
```

What these do:

- `python3-smbus`: Lets Python talk to the X728 via I2C
- `python3-rpi.gpio`: Traditional GPIO control (fallback)
- `gpiod`: Modern GPIO control (primary method, same as official Geekworm scripts)

✅ **Success Check:** The installation should complete without errors

---

Step 4: Create the Script

Now we'll create our power monitoring script:

4.1 Create Project Directory

```bash
mkdir ~/x728_power_monitor
cd ~/x728_power_monitor
```

4.2 Create the Python Script

1. **Create a new file:**

```bash
nano x728_auto_shutdown.py
```

2. **Copy and paste the entire Python script** from the first artifact into this file
3. **Save the file:**
   - Press Ctrl + X
   - Press Y to confirm
   - Press Enter to save

4.3 Make Script Executable

```
bash

chmod +x x728_auto_shutdown.py
```

Step 5: Test the Script

Let's test our script to make sure it works:

5.1 Run Test Mode

```
bash

python3 x728_auto_shutdown.py
```

What you should see:

- A welcome message saying "X728 UPS Power Monitor Starting..."
- "AC Power: OK" if your power is connected
- "Press Ctrl+C to stop monitoring"

5.2 Test Power Loss Detection

1. **With the script running**, carefully **unplug the power adapter** from the X728
2. **You should see:**
   - "⚠ AC POWER LOSS DETECTED!"
   - "This will shut down BOTH the Pi and UPS"
   - A countdown "Shutting down Pi + UPS in 5 seconds..."
   - **Quickly plug the power back in** to cancel the shutdown
3. **If you want to test full shutdown:**
   - Unplug power and **let the countdown finish**
   - Your Pi should shut down safely after 5 seconds
   - **The UPS should also power down completely** (all LEDs off)

- When you plug power back in, the Pi should automatically start

5.3 Stop Test Mode

- **Press** Ctrl + C to stop the script

✅ **Success Check:** The script should detect power loss and start the countdown

---

Step 6: Set Up Automatic Start

Now let's make the script start automatically when your Pi boots:

6.1 Create Service File

```bash
sudo nano /etc/systemd/system/x728-power-monitor.service
```

6.2 Add Service Configuration

Copy and paste this into the file:

```ini
[Unit]
Description=X728 UPS Power Monitor
After=multi-user.target
StartLimitIntervalSec=0

[Service]
Type=simple
Restart=always
RestartSec=1
User=pi
ExecStart=/usr/bin/python3 /home/pi/x728_power_monitor/x728_auto_shutdown.py
ExecStop=/usr/bin/gpioset 0 26=1
ExecStopPost=/bin/sleep 4
TimeoutStopSec=10
KillMode=mixed
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Important changes:

- `ExecStop=/usr/bin/gpioset 0 26=1` - This triggers UPS shutdown during system shutdown
- `ExecStopPost=/bin/sleep 4` - Gives UPS time to register the signal
- `TimeoutStopSec=10` - Prevents systemd from killing the stop process too quickly
- `KillMode=mixed` - Allows proper cleanup

6.3 Save and Enable Service

1. **Save the file:** `Ctrl + X`, then `Y`, then `Enter`
2. **Enable the service:**

```bash
sudo systemctl daemon-reload
sudo systemctl enable x728-power-monitor.service
sudo systemctl start x728-power-monitor.service
```

6.4 Check Service Status

```bash
sudo systemctl status x728-power-monitor.service
```

✅ **Success Check:** Status should show "active (running)" in green

---

Step 7: Monitor and Logs

7.1 View Live Logs

To see what the script is doing in real-time:

```bash
sudo journalctl -u x728-power-monitor.service -f
```

(Press `Ctrl + C` to stop viewing)

7.2 View Log File

The script also saves logs to a file in your project directory:

```bash
tail -f ~/x728_power_monitor/x728_shutdown.log
```

7.3 Log Management Features

✅ **Automatic Log Rotation Built-In:**

- Log file automatically stays under 1000 lines
- Older entries are automatically removed
- No manual maintenance required
- Typically uses less than 100KB of space

7.4 Check if Service is Running

```bash
sudo systemctl is-active x728-power-monitor.service
```

Step 8: Final Testing

8.1 Test Automatic Startup

1. **Reboot your Pi:**

   ```bash
   sudo reboot
   ```

2. **After reboot, check if service started:**

   ```bash
   sudo systemctl status x728-power-monitor.service
   ```

8.2 Test Power Loss Protection

1. **Check the logs to confirm monitoring:**

   ```bash
   sudo journalctl -u x728-power-monitor.service --since "1 minute ago"
   ```

2. **Carefully unplug power** and verify the countdown starts
3. **Plug power back in** to stop the shutdown

Troubleshooting

Problem: "Permission denied" errors

Solution: Make sure you're using `sudo` for system commands:

```bash
sudo systemctl status x728-power-monitor.service
```

Problem: Script can't find GPIO pins

Solution: Check I2C is enabled and X728 is properly connected:

```bash
sudo i2cdetect -y 1
```

Problem: Service won't start

Solution: Check the service file for typos:

```bash
sudo systemctl status x728-power-monitor.service
```

Problem: Script works manually but UPS doesn't power down when run as service

This is the exact issue you experienced!

Root Cause: systemd terminates services before GPIO commands complete during shutdown

Solution: The updated service file now uses `ExecStop` to handle UPS shutdown:

1. **Update your service file** with the new configuration above
2. **Reload systemd:**

   ```bash
   sudo systemctl daemon-reload
   sudo systemctl restart x728-power-monitor.service
   ```

3. **Test the ExecStop command manually:**

   ```bash
   /usr/bin/gpioset 0 26=1
   sleep 4
   ```

Verify the fix:

```bash
sudo systemctl show x728-power-monitor.service -p ExecStop
```

Should show: `ExecStop={ path=/usr/bin/gpioset ; argv[]=/usr/bin/gpioset 0 26=1 ; ... }` **This is the exact issue you experienced!**

Root Cause: systemd terminates services before GPIO commands complete during shutdown

Solution: The updated service file now uses `ExecStop` to handle UPS shutdown:

1. **Update your service file** with the new configuration above
2. **Reload systemd:**

```bash
sudo systemctl daemon-reload
sudo systemctl restart x728-power-monitor.service
```

3. **Test the ExecStop command manually:**

```bash
/usr/bin/gpioset 0 26=1
sleep 4
```

Verify the fix:

```bash
sudo systemctl show x728-power-monitor.service -p ExecStop
```

Should show: `ExecStop={ path=/usr/bin/gpioset ; argv[]=/usr/bin/gpioset 0 26=1 ; ... }`

Problem: False power loss detections

Solution: Increase the debounce time in the script by editing line 15:

```python
DEBOUNCE_TIME = 1.0  # Increase from 0.5 to 1.0 seconds
```

Useful Commands Reference

Service Management

```bash
```

```
# Start the service
sudo systemctl start x728-power-monitor.service

# Stop the service
sudo systemctl stop x728-power-monitor.service

# Restart the service
sudo systemctl restart x728-power-monitor.service

# Check if service is running
sudo systemctl is-active x728-power-monitor.service

# View service status
sudo systemctl status x728-power-monitor.service

# Disable auto-start
sudo systemctl disable x728-power-monitor.service

# Enable auto-start
sudo systemctl enable x728-power-monitor.service
```

Log Viewing

bash

```
# View recent logs
sudo journalctl -u x728-power-monitor.service --since "10 minutes ago"

# View live logs
sudo journalctl -u x728-power-monitor.service -f

# View log file (saved in your project directory with auto-rotation)
tail -f ~/x728_power_monitor/x728_shutdown.log

# Clear logs manually (if needed)
> ~/x728_power_monitor/x728_shutdown.log

# Check log file size
ls -lh ~/x728_power_monitor/x728_shutdown.log
```

Congratulations!

You've successfully set up complete shutdown protection for your Raspberry Pi **and X728 UPS**! Here's what you've accomplished:

✅ **Enabled I2C communication** between Pi and X728
✅ **Created a smart monitoring script** that watches for power loss
✅ **Set up automatic startup** so protection runs at boot
✅ **Tested the system** to ensure it works properly
✅ **Learned useful troubleshooting commands**
✅ **Implemented proper UPS shutdown** to conserve battery

What Happens Now?

🔋 **Normal Operation:** Your Pi runs normally with UPS protection
⚡ **Power Loss:** Both Pi and UPS automatically shut down safely after 5 seconds
🔌 **Power Restored:** Pi automatically turns back on (if AUTO ON jumper is connected)
📝 **Logging:** All events are logged for your review
🛡️ **Battery Conservation:** UPS powers off completely to save battery life

Key Differences from Basic Scripts:

❌ **Regular Linux Shutdown:** Only shuts down Pi, UPS keeps running and drains battery
✅ **Our Enhanced Script:** Properly shuts down both Pi AND UPS, just like the official `x728off` command

This means:

- **Better battery life** - UPS doesn't stay on unnecessarily
- **True "power off"** - Everything shuts down completely
- **Proper power cycling** - Clean restart when power returns

Remember:

- The script runs automatically every time you boot your Pi
- **Logs are automatically managed** - they stay under 1000 lines and ~100KB
- You can always check the status with `sudo systemctl status x728-power-monitor.service`
- Manual log clearing is available but rarely needed thanks to auto-rotation

Your Raspberry Pi is now protected against unexpected power loss with smart log management! 🎉